

Αλγόριθμοι και Πολυπλοκότητα

ΦΡΟΝΤΙΣΤΗΡΙΑΚΕΣ ΑΣΚΗΣΕΙΣ (5)

Διαίρει και βασίλευε

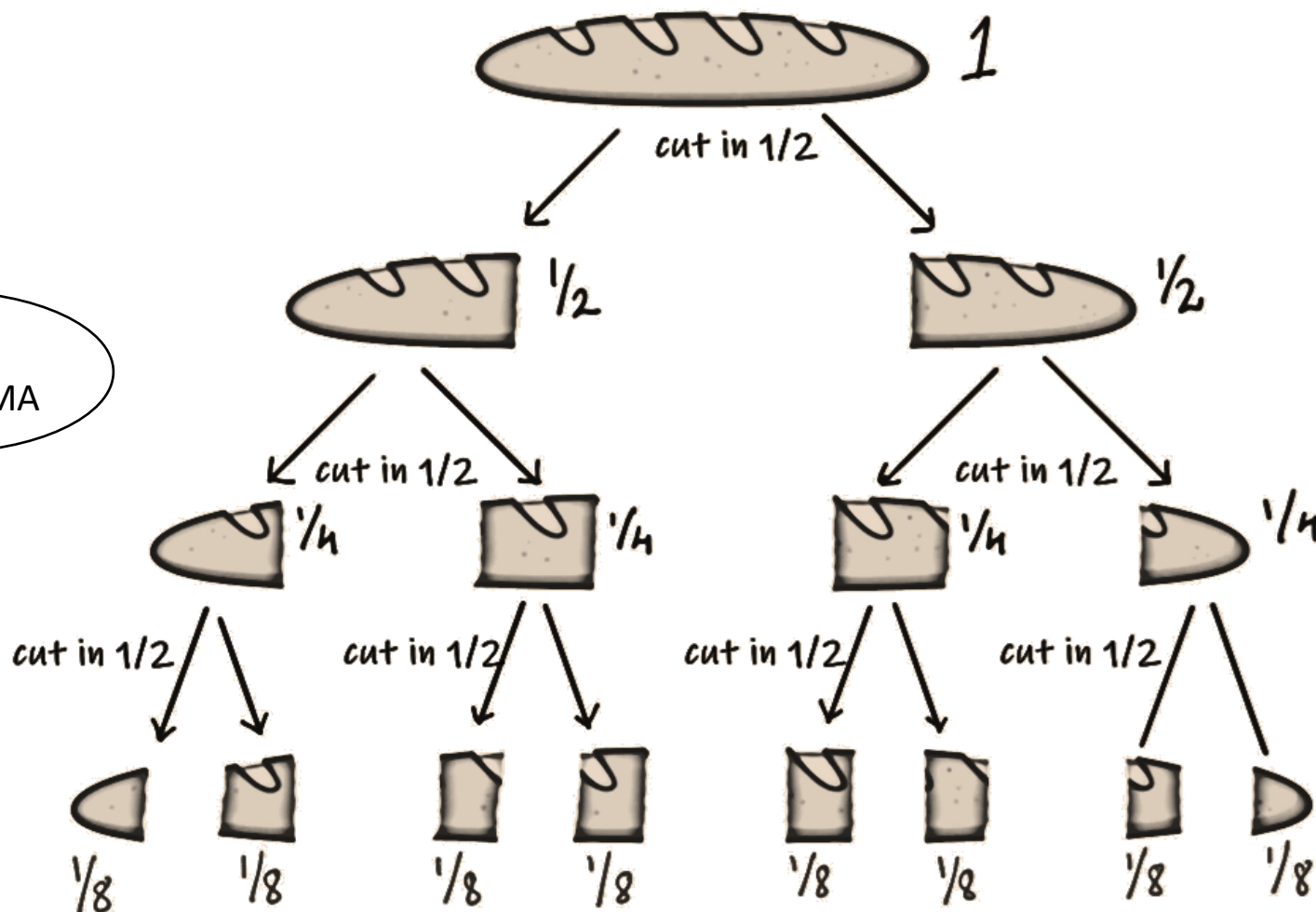
16/4/2024

Έφη Μαλέσιου
Διδακτορική φοιτήτρια

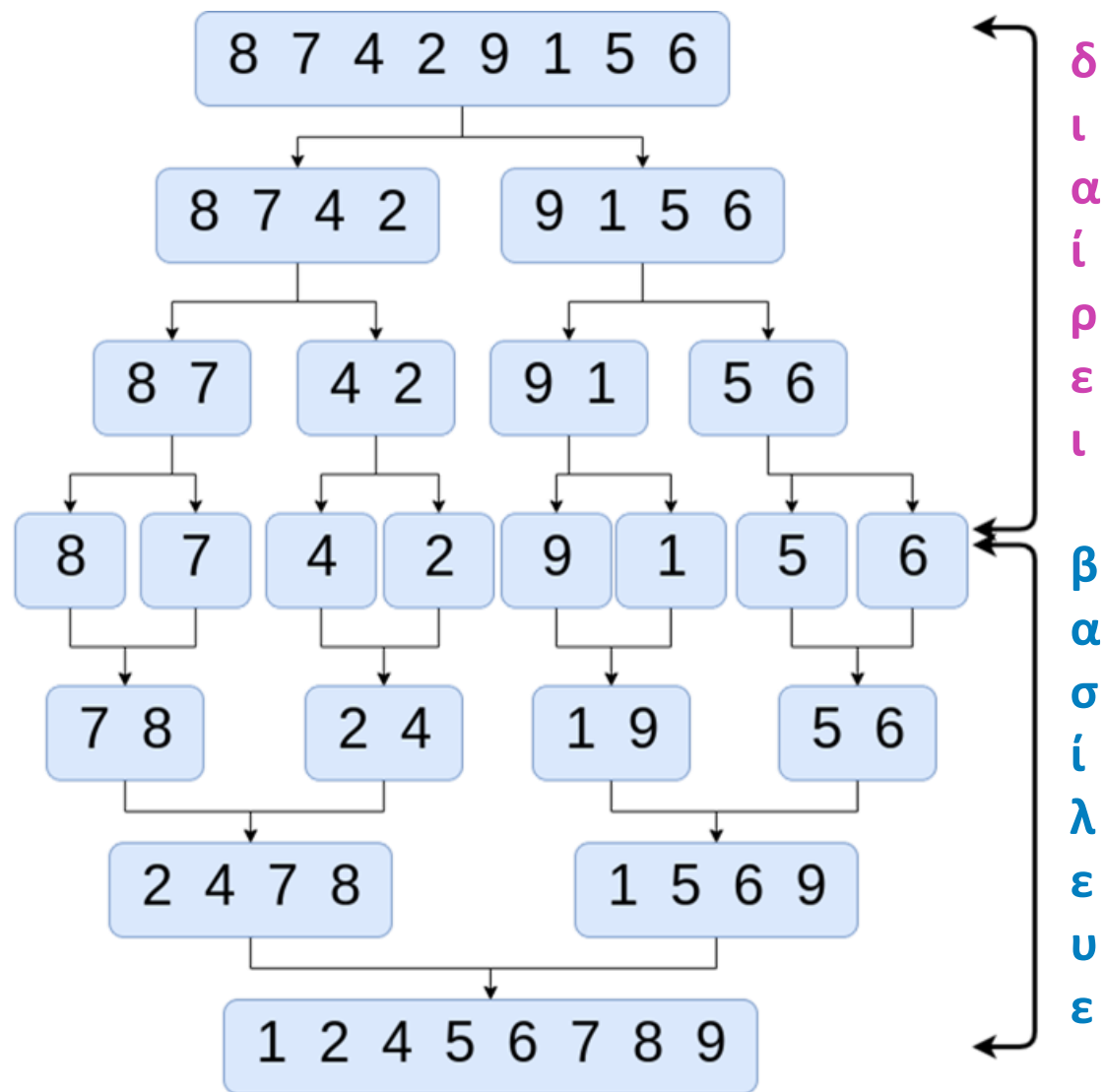
Διαίρει και βασίλευε

ΥΠΟ-
ΠΡΟΒΛΗΜΑ

επίλυση
υπο-προβλήματος

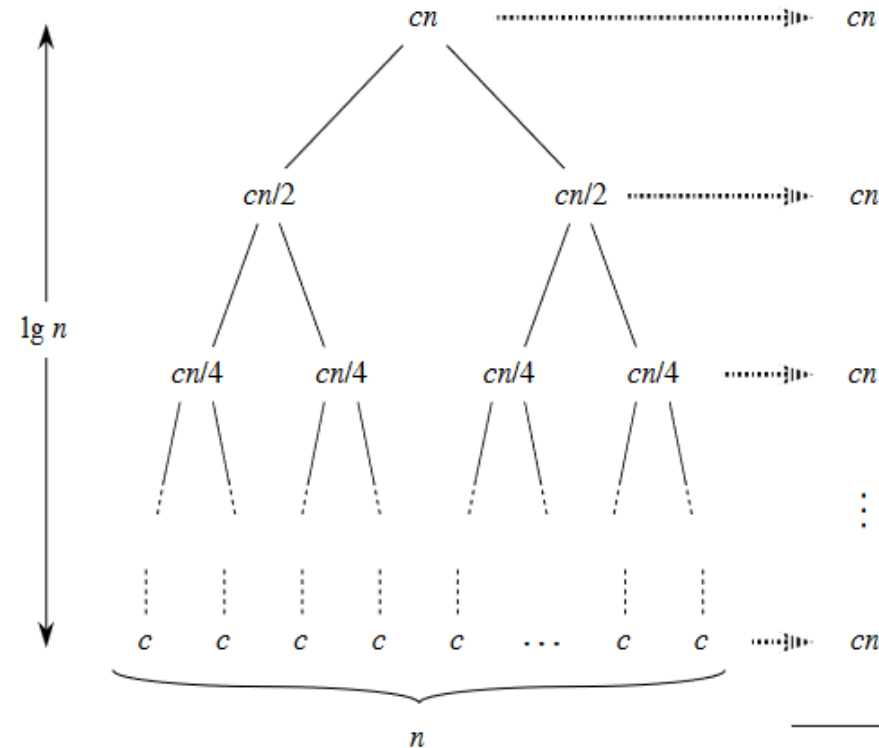
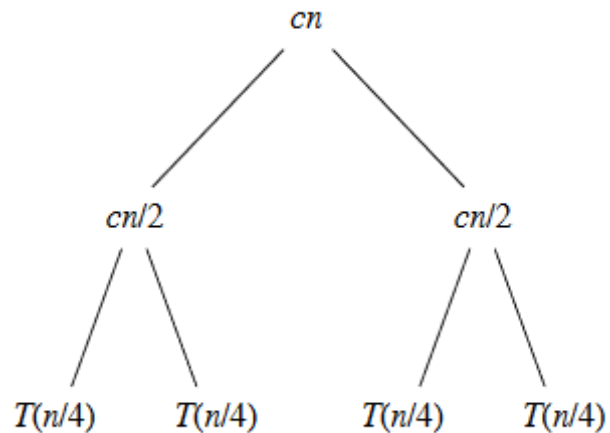
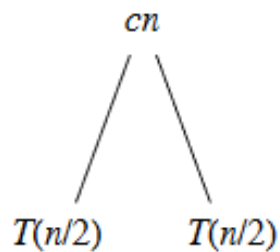


- $T(n)$: χρόνος για πρόβλημα μεγέθους n
- $D(n)$: χρόνος διαίρεσης $\gg \gg$
- $C(n)$: χρόνος συνδυασμού λύσεων $\gg \gg$
- a : αριθμός υπο-προβλημάτων
- n/b : μέγεθος υπο-προβλήματος
- $a \cdot T\left(\frac{n}{b}\right)$: χρόνος επίλυσης των υπο-προβλημάτων
- $$T(n) = \begin{cases} \Theta(1), n \leq c \\ a \cdot T\left(\frac{n}{b}\right) + D(n) + C(n), n > c \end{cases}$$



Ταξινόμηση με Συγχώνευση

Κόστος:



$$(c \cdot n \cdot \lg n + c \cdot n)$$

$\lg n + 1$ επίπεδα

- $\times 2$ υπο-προβλήματος
- $/2$ μέγεθος υπο-προβλήματος

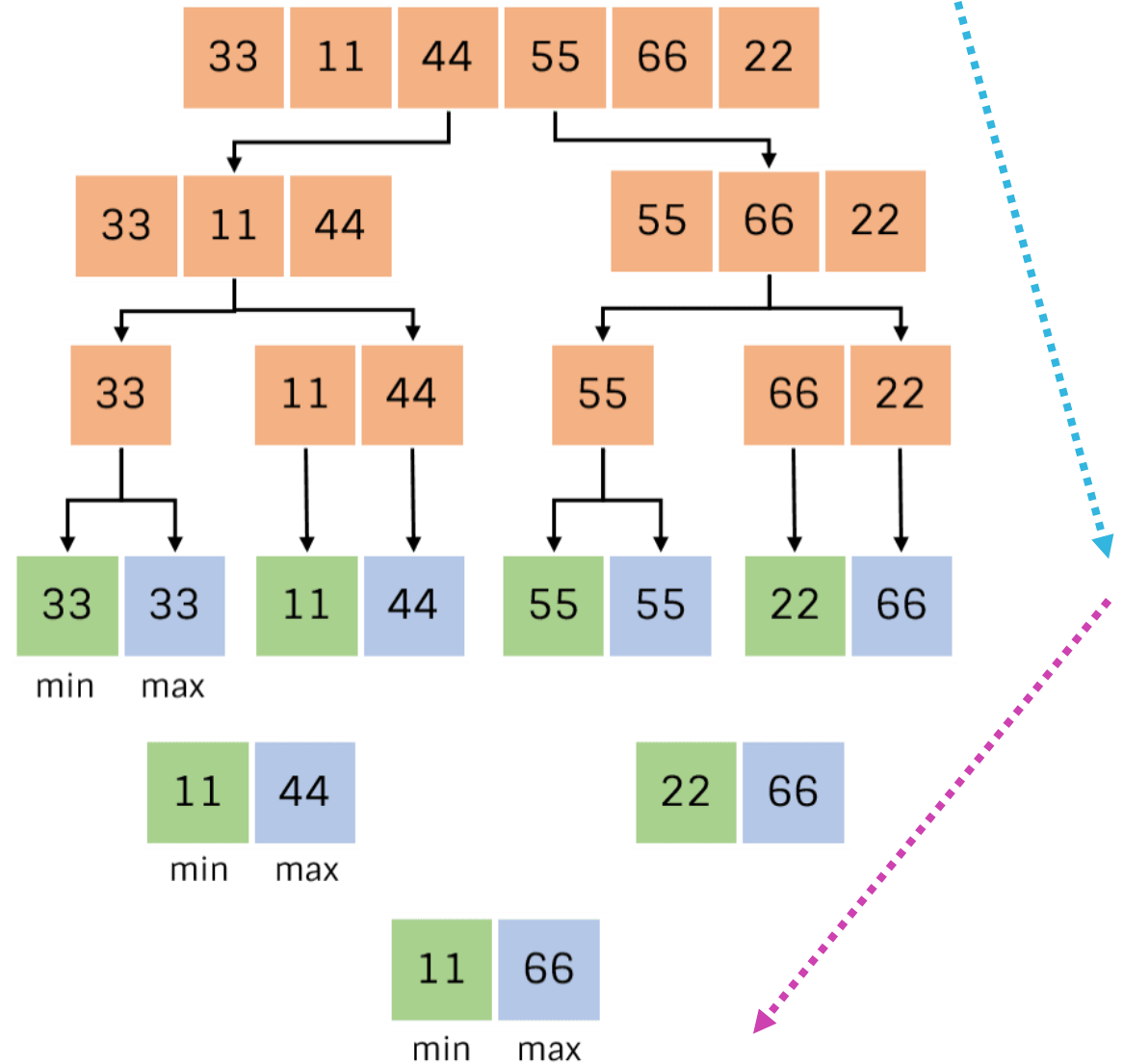
Ταξινόμηση με συγχώνευση ($n=2^a$)

Αριθμός συγκρίσεων ($n=2^k, k \geq 0$):

$$T(n) = \begin{cases} 0, & n = 1 \\ 1, & n = 2 \\ T(n/2) + T(n/2) + 2, & n > 2 \end{cases}$$

$$\begin{aligned} T(n) &= 2T(n/2) + 2 \\ &= 2(2T(n/4) + 2) + 2 \\ &= 4T(n/4) + 4 + 2 \\ &\vdots \\ &= 2^{k-1} T(2) + \sum_{1 \leq i \leq k-1} 2^i \\ &= 2^{k-1} + 2^k - 2 \\ &= (3n/2 - 2) = O(n) \end{aligned}$$

- Καλύτερη / μέση / χειρότερη περίπτωση
- **Άμεση μέθοδος:** $2(n-1) = (2n - 2) \Rightarrow 25\%$ μείωση



Longest Common Prefix ($LCP(S_1, S_2, \dots, S_n)$)

Μέγιστο κοινό πρόθεμα: Δίνεται ένας πίνακας n συμβολοσειρών. Δώστε έναν αποδοτικό αλγόριθμο για την εύρεση του μέγιστου κοινού προθέματος. Δίνεται ότι το μήκος της μεγαλύτερης συμβολοσειράς είναι m .

Παράδειγμα 1:

Input ["leetcode", "leef", "leets", "leeds"]

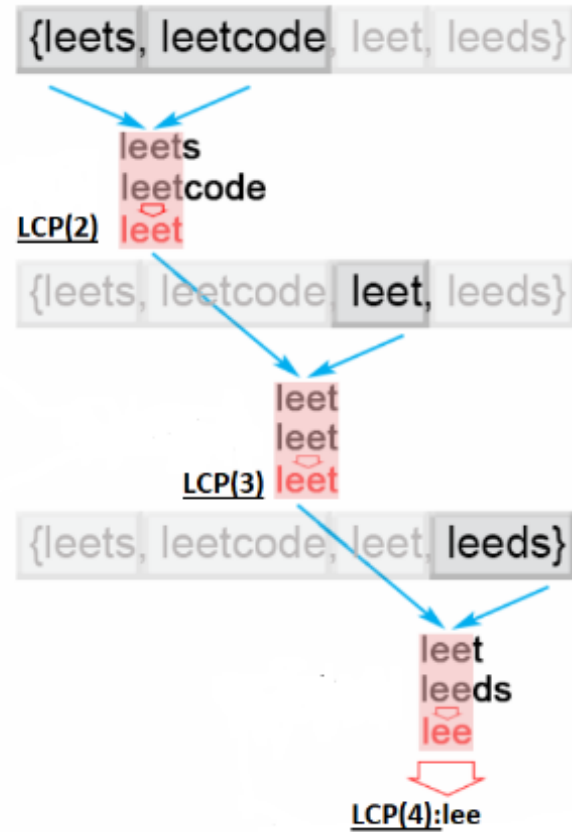
Output LCP: lee

Παράδειγμα 2:

Input ["technique", "technician", "technical", "technology"]

Output LCP: techn

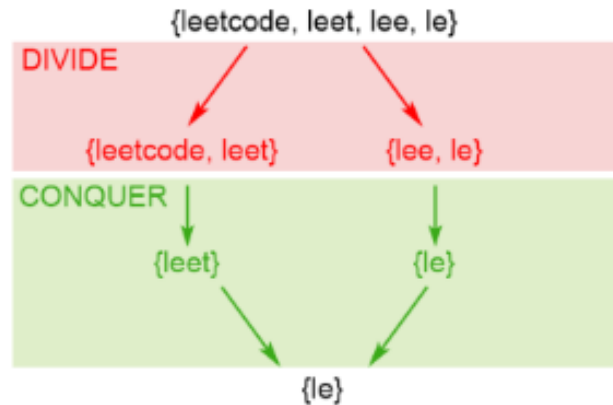
Ωμή Βία: Σε κάθε βήμα εξετάζω μια συμβολοσειρά S_i . Υπολογίζω το $LCP(i)$ συγκρίνοντας την S_i με το $LCP(i - 1)$. $O(nm)$.



Διαίρει και βασίλευε

$LCP(S_1, S_2, \dots, S_n)$: Διαίρει και Βασίλευε

- Διαίρεσε τον πίνακα σε δυο ίσα κομμάτια.
- Αναδρομικά υπολόγισε το $LCP1$ και $LCP2$ των 2 υποπινάκων $LCP(S_1, S_2, \dots, S_{mid})$ και $LCP(S_{mid+1}, S_{mid+2}, \dots, S_n)$ αντίστοιχα.
- Υπολόγισε το $LCP(S_1, S_2, \dots, S_n)$ συγκρίνοντας έναν έναν τους χαρακτήρες των $LCP1$ και $LCP2$.



$$T(n) = 2T(n/2) + O(m) \text{ Οπότε } T(n) = O(mn)$$

Το m λαμβάνεται υπόψη γιατί μπορεί να επιβαρύνει τη διαδικασία, αφού δεν περιορίζεται και ενδέχεται π.χ. να είναι $m = n^2$.

- **Μέθοδος αντικατάστασης**

- υπόθεση για τη μορφή της λύσης
- επιβεβαίωση με επαγωγή
- εύρεση των σταθερών ώστε να ισχύει
- ακριβής ή ασυμπτωτική λύση (άνω & κάτω όρια)
- π.χ. $T(n) = 2T(n/2) + n$ για $n > 1$ και 1 για $n = 1$

- **Ομογενής γραμμική αναδρομική σχέση**

- $T(n) = a_0 + a_1 * T(n-1) + a_2 * T(n-2) + \dots + a_k * T(n-k)$, με $a_0 = 0$
- γραμμική, πεπερασμένος αριθμός όρων, σταθερές τιμές συντελεστών
- προσοχή στις ρίζες με πολλαπλότητα > 1

- **Μέθοδος κυριαρχίας (θεώρημα Master)**

- $f(n)$ vs. $n^{\log_b a} \log^k n$
- Πότε μπορεί να χρησιμοποιηθεί;
- Μπορεί να μετατραπεί σε κατάλληλη μορφή;
- Σε ποια από τις 3 περιπτώσεις ανήκει η αναδρομική εξίσωση;

- **Δένδρο αναδρομής**

- μεγάλη προσοχή στην αιτιολόγηση
- ενδείκνυται περισσότερο για τον σχηματισμό υποθέσεων

- **Επαναληπτική μέθοδος**

- αρκετά εύκολα προκύπτουν λάθη

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

$$T(1) = c$$

$$\alpha \geq 1, b > 1, c > 0, f(n) > 0$$

ασυμπτωτικά

η $f(n)$ πρέπει να είναι:

1. Εάν $f(n) = O(n^{\log_b a - \epsilon})$ για κάποια σταθερά $\epsilon > 0$ τότε $T(n) = \Theta(n^{\log_b a})$.
2. Εάν $f(n) = \Theta(n^{\log_b a})$ τότε $T(n) = \Theta(n^{\log_b a} \lg n)$.
3. Εάν $f(n) = \Omega(n^{\log_b a + \epsilon})$ για κάποια σταθερά $\epsilon > 0$ και εάν $af(\frac{n}{b}) \leq cf(n)$ για κάποια σταθερά $c < 1$ και όλα τα αρκετά μεγάλα n τότε $T(n) = \Theta(f(n))$.

* πολυωνυμικά μικρότερη

* πολυωνυμικά μεγαλύτερη

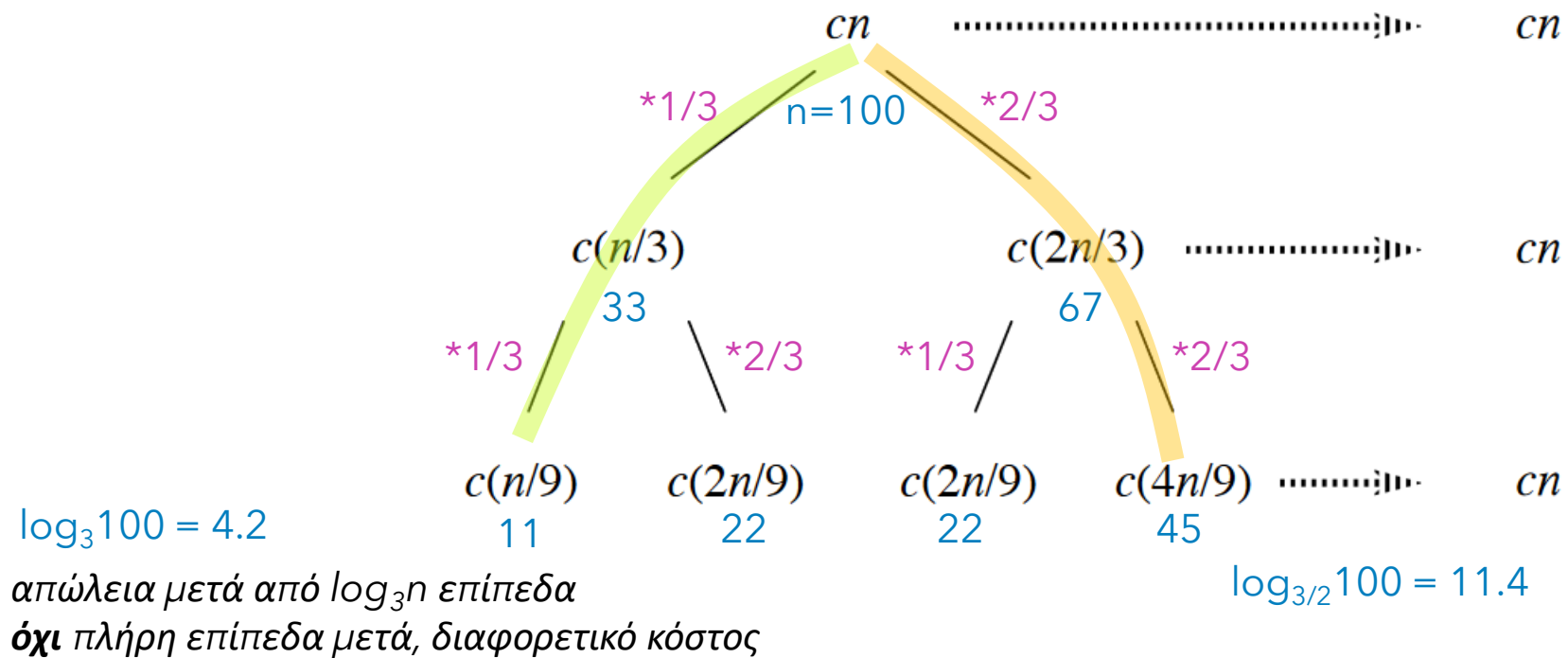
Πρέπει:

- η $T(n)$ να είναι μονότονη
- η $f(n)$ πολυωνυμική / όροι της μορφής $(k \cdot n^d)$ με $k, d \in \mathbb{R}^+$, π.χ. όχι 2^n
 - 4. Όμως, αν $f(n) = \Theta(n^{\log_b a} \log^k n)$, $k \geq 0$, $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$
- το b να μπορεί να εκφραστεί ως σταθερά

Δένδρο αναδρομής

$$T(n) = T(n/3) + T(2n/3) + \Theta(n)$$

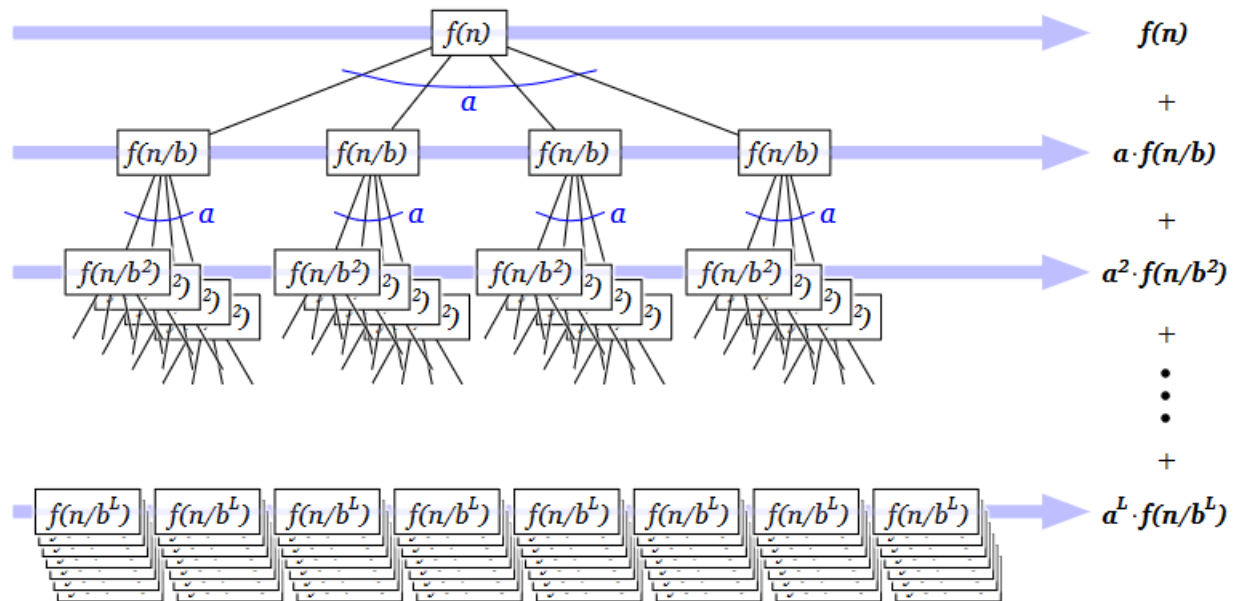
{αριστερά} {δεξιά}



Επίλυση αναδρομικών εξισώσεων

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

$$T(1) = c$$



Βασική περίπτωση: $T(1)$

* πότε το μέγεθος του υπο-προβλήματος γίνεται 1;

στο επίπεδο k :

$$n/b^k = 1 = n / n \Rightarrow n = b^k$$

$$\Rightarrow \log(n) = \log(b^k)$$

$$\Rightarrow k = \log(n) / \log(b) = \mathbf{\log_b(n)}$$

* πόσα υπο-προβλήματα υπάρχουν σε αυτό το επίπεδο (φύλλα);

$$a^k = a^{\log_b(n)} = n^{\log_b(a)}$$

κόστος φύλλων =

#υπο-προβλημάτων * στοιχειώδες κόστος =

$$a^{\log_b(n)} * T(1) = a^{\log_b(n)} * c$$

$$h = \log_b(n) + 1$$

υπόθεση: n = δύναμη του b

επαγωγή: $T(b^0 * n) \Rightarrow T(b^1 * n)$

Δένδρο αναδρομής

$$T(n) = \begin{cases} T(1) = 7 \\ 2T\left(\frac{n}{2}\right) + 5n^2 \end{cases}$$

Συνολικό κόστος

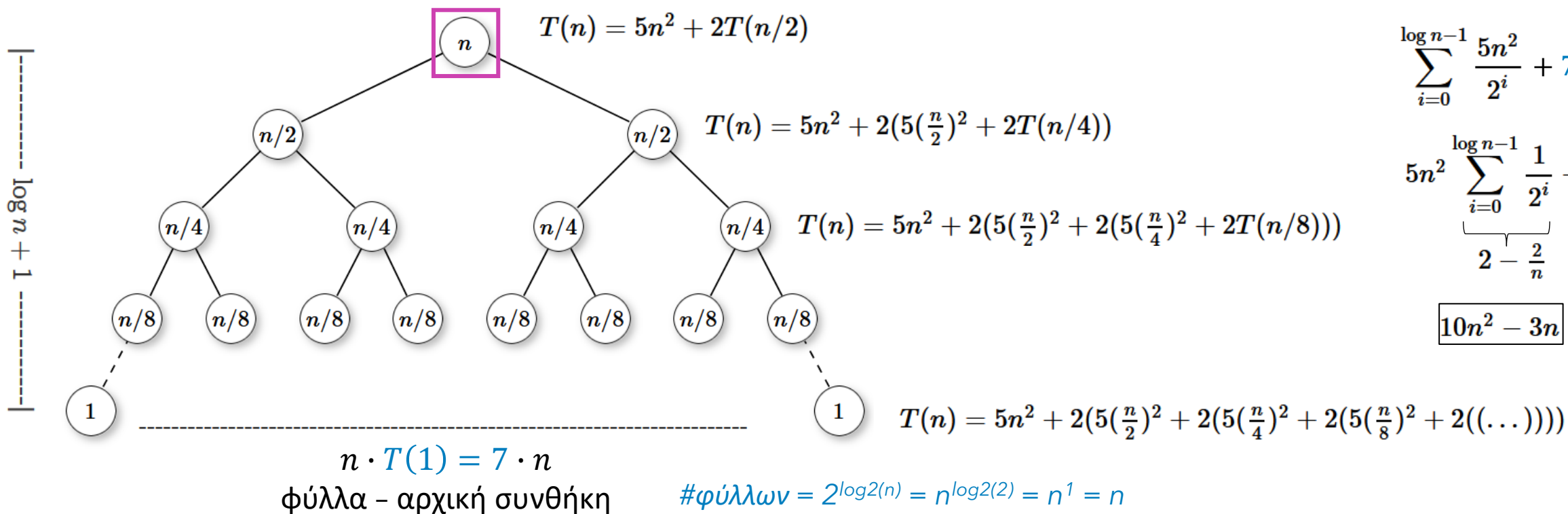
$$\sum_{i=0}^{\log_b n - 1} a^i f\left(\frac{n}{b^i}\right) + a^{\log_b n} \cdot T(1)$$

κόστος φύλλων

$$\sum_{i=0}^{\log n - 1} \frac{5n^2}{2^i} + 7n$$

$$5n^2 \underbrace{\sum_{i=0}^{\log n - 1} \frac{1}{2^i}}_{2 - \frac{2}{n}} + 7n$$

$$10n^2 - 3n$$



Δένδρο αναδρομής - Ακριβής λύση

Ρίζες χαρακτηριστικής εξίσωσης:

- Δευτεροβάθμια εξίσωση – με 2 πραγματικές ρίζες:

- 2 διαφορετικές ρίζες ($x_1 \neq x_2$):

- $T_n = \lambda_1 x_1^n + \lambda_2 x_2^n$

- 1 διπλή ρίζα ($x_1 = x_2$):

- $T_n = \lambda_1 x_1^n + \lambda_2 n x_2^n$

- λ_1, λ_2 από τις αρχικές συνθήκες

- Τριτοβάθμια εξίσωση – με 3 πραγματικές ρίζες:

- 3 διαφορετικές ρίζες ($x_1 \neq x_2, x_1 \neq x_3, x_2 \neq x_3$):

- $T_n = \lambda_1 x_1^n + \lambda_2 x_2^n + \lambda_3 x_3^n$

- 1 διπλή ρίζα ($x_1 = x_2 \neq x_3$):

- $T_n = \lambda_1 x_1^n + \lambda_2 n x_2^n + \lambda_3 x_3^n$

- 1 τριπλή ρίζα ($x_1 = x_2 = x_3$):

- $T_n = \lambda_1 x_1^n + \lambda_2 n x_2^n + \lambda_3 n^2 x_3^n$

- $\lambda_1, \lambda_2, \lambda_3$ από τις αρχικές συνθήκες

$$F(n) = \begin{cases} 0, n = 0 \\ 1, n = 1 \\ F(n-1) + F(n-2), n \geq 2 \end{cases}$$

$$\text{Θέτουμε: } F(n) = x^n$$

$$F(n) - F(n-1) - F(n-2) = 0 \Rightarrow x^n - x^{n-1} - x^{n-2} = 0 \Rightarrow x^{n-2}(x^2 - x - 1) = 0$$

$$\text{Διακρίνουσα } (x^2 - x - 1) = 1 + 4 = 5 \Rightarrow x_1 = \frac{1+\sqrt{5}}{2}, x_2 = \frac{1-\sqrt{5}}{2}$$

$$\text{Συνεπώς: } F_n = \lambda_1 x_1^n + \lambda_2 x_2^n$$

$$F_0 = \lambda_1 x_1^0 + \lambda_2 x_2^0 = 0 \Rightarrow \lambda_1 = -\lambda_2$$

$$F_1 = \lambda_1 x_1^1 + \lambda_2 x_2^1 = \lambda_1 x_1^1 - \lambda_1 x_2^1 = 1 \Rightarrow \lambda_1 = \frac{1}{\sqrt{5}}, \lambda_2 = -\frac{1}{\sqrt{5}}$$

$$F_n = \frac{1}{\sqrt{5}} \left(\frac{1+\sqrt{5}}{2} \right)^n - \frac{1}{\sqrt{5}} \left(\frac{1-\sqrt{5}}{2} \right)^n, \text{ εκθετική πολυπλοκότητα}$$

$$T(n) = \begin{cases} 3, n = 0 \\ 8, n = 1 \\ 4T(n-1) - 4T(n-2), n \geq 2 \end{cases}$$

$$\text{Θέτουμε: } T(n) = x^n$$

$$T(n) - 4T(n-1) + 4T(n-2) = 0 \Rightarrow$$

$$x^n - 4x^{n-1} + 4x^{n-2} = 0 \Rightarrow$$

$$x^{n-2}(x^2 - 4x + 4) = 0 \Rightarrow$$

$$(x - 2)^2 = 0$$

$$x_1 = 2, x_2 = 2$$

Συνεπώς, επειδή η ρίζα 2 έχει πολλαπλότητα > 1 :

$$T_n = \lambda_1 x_1^n + \lambda_2 n x_2^n = \\ \lambda_1 2^n + \lambda_2 n 2^n$$

$$T(n) = \begin{cases} 3, n = 0 \\ 8, n = 1 \\ 4T(n-1) - 4T(n-2), n \geq 2 \end{cases}$$

Χρησιμοποιώντας τις αρχικές συνθήκες:

$$T_0 = 3 = \lambda_1 2^0 + \lambda_2 \cdot 0 \cdot 2^0 \Rightarrow \lambda_1 = 3$$

$$T_1 = 8 = \lambda_1 2^1 + \lambda_2 \cdot 1 \cdot 2^1 = 2 \cdot 3 + 2\lambda_2 \Rightarrow \lambda_2 = 1$$

$$T_n = 3 \cdot 2^n + 1 \cdot n \cdot 2^n = 3 \cdot 2^n + n \cdot 2^n = (3 + n) \cdot 2^n$$

$$\Theta(n \cdot 2^n)$$

$$T(n) = T(n-1) + 2^n, \quad T_0 = 5$$

$$T_n = T_{n-1} + 2^n, \quad T_0 = 5$$

$$T_1 - T_0 = 2^1$$

$$T_2 - T_1 = 2^2$$

$$T_3 - T_2 = 2^3$$

...

$$T_n - T_{n-1} = 2^n$$

Προσθέτοντας κατά μέλη:

$$T_n - T_0 = \sum_{i=1}^n 2^i = 2^{n+1} - 2$$

$$\sum_{k=0}^n 2^k = 2^{n+1} - 1$$

$$T_n - 5 = 2^{n+1} - 2$$

$$T_n = 2^{n+1} + 3$$

$$2^{n+1} = 2^n \cdot 2^1 = 2 \cdot 2^n \Rightarrow \Theta(2^n)$$

1.Μια ομάδα προγραμματιστών εργάζεται για την επίλυση ενός υπολογιστικού προβλήματος P και έχει δημιουργήσει 3 διαφορετικούς αλγορίθμους «διαίρει και βασίλευε».

- Τον αλγόριθμο A_1 που διασπά το αρχικό πρόβλημα μεγέθους n σε 4 υποπροβλήματα μεγέθους $n/4$, τα επιλύει και στη συνέχεια συνθέτει τις λύσεις τους σε χρόνο $12n$.
- Τον αλγόριθμο A_2 που διασπά το αρχικό πρόβλημα μεγέθους n σε 3 υποπροβλήματα μεγέθους $n/3$, τα επιλύει και στη συνέχεια συνθέτει τις λύσεις τους σε χρόνο $n^{7/6}$.
- Τον αλγόριθμο A_4 που διασπά το αρχικό πρόβλημα μεγέθους n σε 27 υποπροβλήματα μεγέθους $n/3$, τα επιλύει και στη συνέχεια συνθέτει τις λύσεις τους σε χρόνο $n^{11/12}$.

Οι προγραμματιστές ενδιαφέρονται για την ασυμπτωτική πολυπλοκότητα καθενός από τους παραπάνω 3 αλγορίθμους και ζητούν τη βοήθειά σας.

Συγκεκριμένα σας ζητούν:

(A) Να γράψετε τις αναδρομικές εξισώσεις που δίνουν τον χρόνο εκτέλεσης των αλγορίθμων A_1 , A_2 και A_4 και στη συνέχεια να τις επιλύσετε με χρήση του Θεωρήματος της Κυριαρχίας (Master Theorem).

(B) Να προσδιορίσετε ποιος είναι ο ασυμπτωτικά αποδοτικότερος αλγόριθμος που επιλύει το υπολογιστικό πρόβλημα P και να δικαιολογήσετε την απάντησή σας.

A) Η αναδρομική εξίσωση που δίνει τον χρόνο εκτέλεσης του αλγορίθμου A_1 είναι η

$$T_1(n) = 4T_1(n/4) + 12n.$$

Δηλαδή είναι της μορφής $T(n) = aT(n/b) + f(n)$ με $a = 4$, $b = 4$ και $f(n) = 12n$.

Έχουμε ότι $\log_b a = \log_4 4 = 1$, οπότε $f(n) = 12n = \Theta(n) = \Theta(n^1) = \Theta(n^{\log_b a})$.

Συνεπώς, σύμφωνα με την 2^η περίπτωση του Θεωρήματος της Κυριαρχίας, η λύση της αναδρομικής εξίσωσης είναι $T_1(n) = \Theta(n \log n)$.

Η αναδρομική εξίσωση που δίνει τον χρόνο εκτέλεσης του αλγορίθμου A_2 είναι η

$$T_2(n) = 3T_2(n/9) + n^{7/6}.$$

Δηλαδή είναι της μορφής $T(n) = aT(n/b) + f(n)$ με $a = 3$, $b = 9$ και $f(n) = n^{7/6}$.

Έχουμε ότι $\log_b a = \log_9 3 = \frac{\log_3 3}{\log_3 9} = 1/2$, οπότε

$f(n) = n^{7/6} = \Omega(n^{1/2+\varepsilon}) = \Omega(n^{\log_b a + \varepsilon})$, για $\varepsilon = 2/3$. Επίσης,

$a \cdot f(n/b) = 3 \cdot (n/9)^{7/6} = 3 \cdot n^{7/6} / 3^{7/3} = (1/3^{4/3})f(n) \leq c \cdot f(n)$, για οποιαδήποτε

σταθερά $1/3^{4/3} \leq c < 1$. Συνεπώς, σύμφωνα με την 3^η περίπτωση του Θεωρήματος

της Κυριαρχίας, η λύση της αναδρομικής εξίσωσης είναι $T_2(n) = \Theta(n^{7/6})$.

Η αναδρομική εξίσωση που δίνει τον χρόνο εκτέλεσης του αλγορίθμου A_4 είναι η

$$T_4(n) = 27T_4(n/9) + n^{11/12}.$$

Δηλαδή είναι της μορφής $T(n) = aT(n/b) + f(n)$ με $a = 27$, $b = 9$ και $f(n) = n^{11/12}$.

Έχουμε ότι $\log_b a = \log_9 27 = 3/2$, οπότε $f(n) = n^{11/12} = O(n^{\log_b a - \varepsilon})$, για $\varepsilon = 7/12$.

Συνεπώς, σύμφωνα με την 1^η περίπτωση του Θεωρήματος της Κυριαρχίας, η λύση της αναδρομικής εξίσωσης είναι $T_4(n) = \Theta(n^{3/2})$.

B) Εύκολα μπορούμε να διαπιστώσουμε ότι ο χρονικά αποδοτικότερος αλγόριθμος είναι ο αλγόριθμος A_1 .

$$(A_2, A_4): \frac{3}{2} > \frac{7}{6} \Rightarrow n^{\frac{3}{2}} > n^{\frac{7}{6}}$$

$$(A_2, A_1): \lim_{n \rightarrow \infty} \frac{n \log n}{n^{\frac{7}{6}}} = \lim_{n \rightarrow \infty} \frac{\log n}{\frac{1}{n^{\frac{1}{6}}}}$$

$$= (\text{Κανόνας } L'Hospital) \lim_{n \rightarrow \infty} \frac{\frac{1}{n \ln 2}}{\frac{1}{6} n^{-\frac{5}{6}}} = \lim_{n \rightarrow \infty} \frac{6n^{\frac{5}{6}}}{\ln 2 n^{\frac{5}{6}}} = 0 \xrightarrow{n \rightarrow \infty}$$

$$n^{\frac{7}{6}} > n \log n \Rightarrow A_1 \text{ πιο αποδοτικός}$$

Δείξτε με τη βοήθεια της μαθηματικής επαγωγής ότι όταν το n είναι ακριβής δύναμη του 2, η λύση της αναδρομής

$$T(n) = \begin{cases} 2 & \text{αν } n = 2 \\ 2T(n/2) + n & \text{αν } n = 2^k, \quad k > 1 \end{cases}$$

είναι $T(n) = n \log n$.

Λύση.

Έστω $n = 2^k$.

Για $k = 1$ ισχύει.

Υποθέτουμε ότι ισχύει για $k = m$:

$$2^m \log 2^m = 2T\left(\frac{2^m}{2}\right) + 2^m$$

Θα δείξουμε ότι ισχύει για $k = m + 1$.

Έχουμε

$$\begin{aligned} 2T\left(\frac{2^{m+1}}{2}\right) + 2^{m+1} &= 2T(2^m) + 2^{m+1} \\ &= 2(2T\left(\frac{2^m}{2}\right) + 2^m) + 2^{m+1} \\ &\stackrel{(5)}{=} 2(2^m \log 2^m) + 2^{m+1} \\ &= 2^{m+1} \log 2^m + 2^{m+1} \\ &= 2^{m+1} \log 2^{m+1} \end{aligned}$$

$$T(n) = 2T\left(\frac{n}{2}\right) + n \log n$$

$$a = 2 \geq 1$$

$$b = 2 \geq 2$$

$$n^{\log_b a} = n^{\log_2 2} = n^1 = n$$

$$\begin{aligned} f(n) &= n \log n > 0, \text{ ασυμπτωτικά} \\ &= \Theta(n^{\log_b a} \log^k n) = \Theta(n^{\log_2 2} \log^1 n) \\ &= \Theta(n \log n) \end{aligned}$$

Θ. κυριαρχίας - περίπτωση 4:

$$T(n) = \Theta(n^{\log_2 2} \log^{1+1} n) = \Theta(n \log^2 n)$$

$$\begin{aligned}
 T(n) &= T(\sqrt{n}) + 1 \\
 T(1) &= O(1) \\
 \Theta(&;)
 \end{aligned}$$

$$\begin{aligned}
 m &= \log n \\
 S(m) &= T(2^m) \\
 T(2^m) &= T\left(2^{\frac{m}{2}}\right) + 1 \\
 S(m) &= S\left(\frac{m}{2}\right) + 1 \\
 \alpha &= 1 \geq 1 \\
 b &= 2 \geq 2 \\
 f(m) = 1 = m^0 &= \Theta(m^{\log_b a}) = \Theta(m^{\log_b 1})
 \end{aligned}
 \quad \left(\begin{array}{l} 2^m = 2^{\log n} = n^{\log 2} = n, \\ \frac{m}{2} = 2^{\frac{\log n}{2}} = 2^{\log n^{0.5}} = (n^{0.5})^{\log 2} = \sqrt{n} \end{array} \right)$$

Θ. κυριαρχίας - περίπτωση 2:

$$S(m) = \Theta(m^{\log_2 1} \log m) = \Theta(\log m) \Rightarrow T(n) = \Theta(\log \log n)$$

ΣΥΝΕΧΕΙΑ ΣΤΟ ΕΠΟΜΕΝΟ ΦΡΟΝΤΙΣΤΗΡΙΟ...



Πηγές Φ3:

Σημειώσεις / διαλέξεις μαθήματος, προτεινόμενα βιβλία, υλικό μαθήματος
προηγούμενων ετών, SMA5503